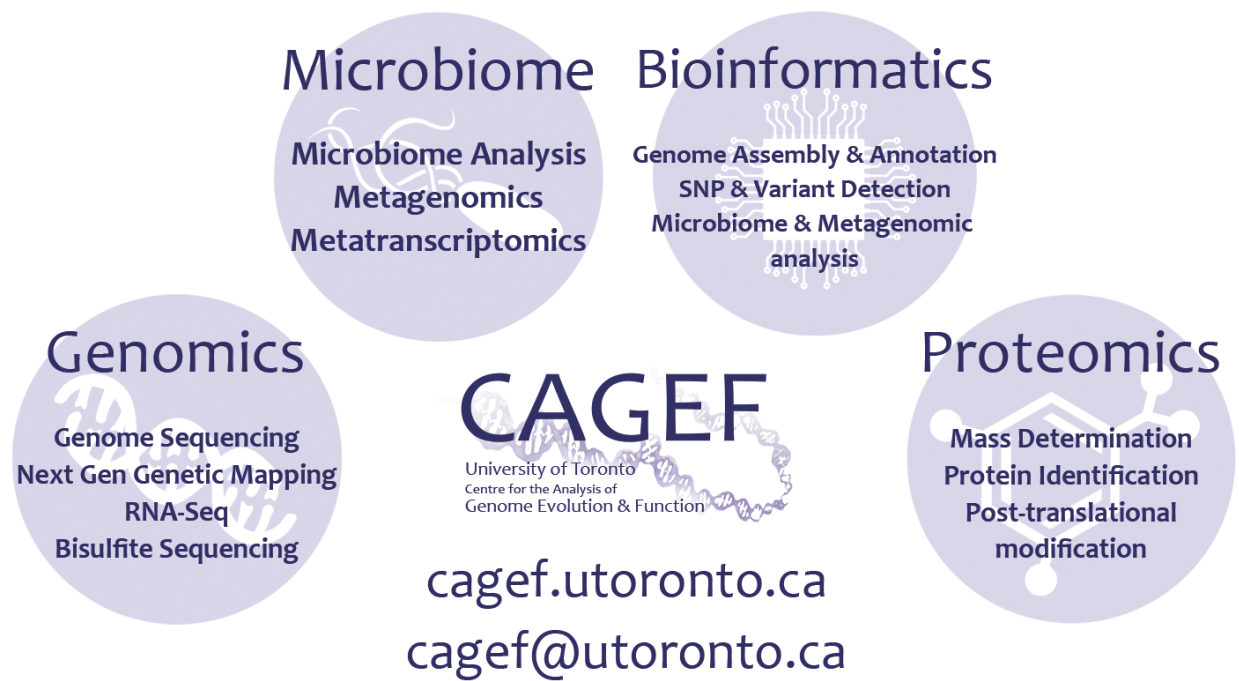




CSB1021HF LEC0131

FUNDAMENTALS OF GENOMIC DATA SCIENCE

0.0.0 Module 1: Exploring Genomic File Formats

0.1.0 About Fundamentals of Genomic Data Science

Fundamentals of Genomic Data Science is brought to you by the **Centre for the Analysis of Genome Evolution & Function (CAGEF)** bioinformatics training initiative. This course was developed based on feedback on the needs and interests of the Department of Cell & Systems Biology and the Department of Ecology and Evolutionary Biology.

The structure of this course is a “code-along”, hands-on style! A few hours prior to each lecture, materials will be made available for download at QUERCUS (<https://q.utoronto.ca/>). The teaching materials will consist of a weekly PDF that you can use to follow along with the instructor along with any datasets that you’ll need to complete the module. This learning approach will allow you to spend the time coding and not taking notes!

As we go along, there will be some in-class challenge questions for you to solve. Post lecture assessments will also be available for each module, building upon the concepts learned in class (see syllabus for grading scheme and percentages of the final mark).

0.1.1 Where is this course going?

We’ll take a blank slate approach here to learning genomic data science and assume you know nothing about programming or working directly with next generation sequencing data. From the beginning of this course to the end we want to guide you from potential scenarios like:

- You don’t know what to do with a set of raw sequencing files fresh from a facility like CAGEF.

- You've been handed a legacy pipeline to analyse your data or maintain for the lab, but you don't know what it runs or how.
- You plan on generating high-throughput data but there are no bioinformaticians around to help you out.

and get you to the point where you can:

- Recognize the basic tools in sequence analysis.
- Plan and write your own data analysis pipelines.
- Explain your data analysis methods to labmates, supervisors, and other colleagues.

0.1.2 How do we get there?

In the first half of this course, we'll focus on how to generate analysis pipelines using the Galaxy platform – a user-friendly graphical interface that provides access to common sequence analysis tools. After we are comfortable with these tools, we'll look at life through the lens of a command-line interface. It is here that we will learn the basics of file manipulation and how to program scripts that can carry out multiple tasks for us. From there we'll revisit tools from the first half and learn skills to make your data analysis journey easier.

0.2.0 Goals of the module

1. Set up your computer for this course.
2. Learn how to download genomic data.
3. Explore the formats of common genomic data.
4. Learn how to manipulate data files in a text editor.
5. Learn how to use the Broad Institute's Integrative Genomics Viewer (IGV).

0.3.0 Pre-class readings or Coursera modules

Each week we strongly encourage you to read any assigned materials or complete the assigned Coursera modules **before** class. These are meant to provide you with sufficient background material on each week's module so that we can focus on the act of "doing" something with that data rather than spend a lot of time on the origins of it. You'll find a section outlining the next set of Coursera modules and readings **at the end** of each module.

0.3.1 Go to www.coursera.org and sign up for an account with your e-mail.

0.3.2 Search the following course and enroll to audit but do not start:

- Command Line Tools for Genomic Data Science, Johns Hopkins University.
 - We will cover related material in Modules 4-7.

0.4.0 Setting up your working directory

We suggest that you create a new directory (folder) for this course directly off your root directory called "**FGDS**". This might look like "C:/" or "/User/xyzName/". Working from your root directory is not necessary, but it will make some of the aspects of the course a little easier to manage. For MacOS users, we suggest you create this as a subfolder in your **user** directory.

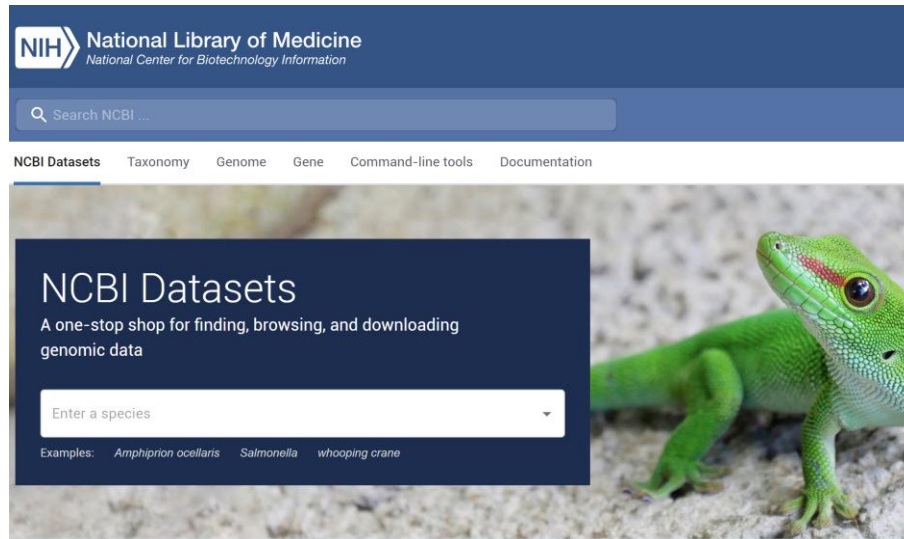
- 0.4.1 Within this directory, create another directory called "**Module1**". This is where we will store the data used in this week's module.
- 0.4.2 Create a subdirectory called "**downloads**" to store the initial files as we download them before decompressing and working with them in later steps.

1.0.0 Obtaining Genomic Data from NCBI

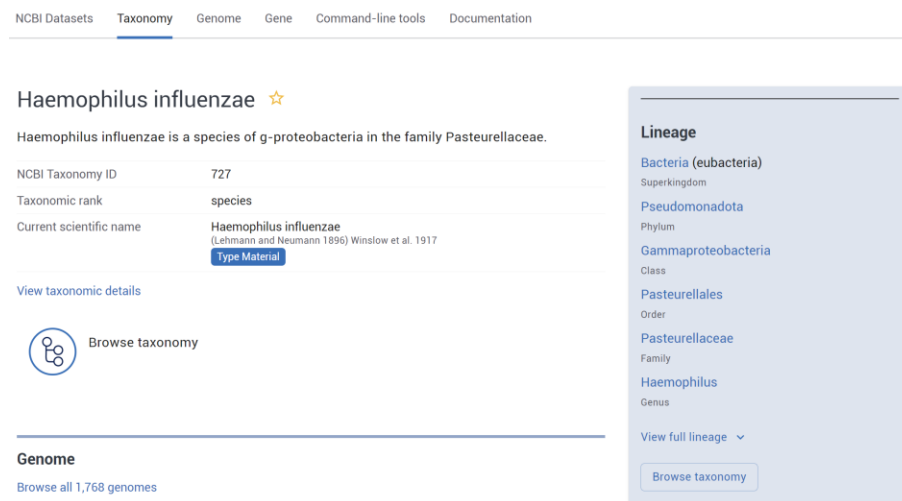
1.1.0 Locating genomic data

Navigating NCBI can, at times, be a daunting experience. Today we'll start with locating genomic data using the *Haemophilus influenzae* str. KW20 genome as an example.

- 1.1.1 Go to <https://www.ncbi.nlm.nih.gov/datasets/>. Note that most NCBI pages have a near the top of the page. This box can be used to search a wide range of different data.



- 1.1.2 Search "*Haemophilus influenzae*" in the (omit the quotation marks). The search results will take you over to the **Taxonomy** tab located on the page.



- 1.1.3 On this page you'll find the taxonomic information for this species which may include a reference genome, and a link to the sequenced genomes for *H. influenzae*.
- 1.1.4 Find the which will take you to a page with all the sequenced strains of *H. influenzae* that are available on NCBI. You'll notice you are now on the **Genome** tab of the page.
- 1.1.5 In the **Filters** submenu, search within results for

Genome

Download a genome data package including genome, transcript and protein sequence, annotation and a data report

Selected taxa
Haemophilus influenzae Enter one or more taxonomic names

Filters Rd KW20

STATUS

☐ Reference genomes

☐ Annotated genomes

☐ Annotated by NCBI RefSeq

☐ Annotated by GenBank submitter

☐ Exclude atypical genomes

☐ Metagenome-assembled genomes (MAGs)

SEARCH WITHIN RESULTS

Enter taxon name or modifier, assembly name or submitter

ASSEMBLY LEVEL

contig scaffold chromosome complete

YEAR RELEASED

1980 2023

Download Select columns 1 genome Rows per page 20 1-1 of 1

☐ Assembly	GenBank	RefSeq	Scientific name	Modifier	Annotation	Action
☐ ASM2730v1	GCA_000027305.1	GCF_000027305.1	Haemophilus influenzae Rd KW...	Rd KW20 (strain)	NCBI RefSeq Submitter	

Now we've located our genome of interest. From here there are several pieces of information we can use to learn about our genome of interest such as the origins of the biological sample, size of the sequenced genome, and a list of coding sequence locations.

1.2.0 Downloading genome assembly files

What we are ultimately interested in from this report is the genome assembly itself which will hold files that include annotations of features as well as DNA, RNA and protein sequences if applicable.

- 1.2.1 From the genome assembly report table, under the Assembly column, click on the accession **ASM2730v1** hyperlink which will take us to the assembly report.

Genome assembly ASM2730v1

Download datasets curl

Status: RefSeq GCF_000027305.1 is suppressed

This record was removed by RefSeq staff. Please contact info@ncbi.nlm.nih.gov for further details. Reason: many frameshifted proteins

	Actions
NCBI RefSeq assembly	GCF_000027305.1 (suppressed)
Submitted GenBank assembly	GCA_000027305.1
Taxon	Haemophilus influenzae Rd KW20
Strain	Rd KW20
Submitter	TIGR
Date	Dec 22, 1999

View the legacy Assembly page

Additional genomes

Browse all Haemophilus influenzae genomes (1321)

BioProject

PRJNA219
Human-specific organism that causes a variety of diseases (avirulent strain)

Publications

Showing 5 of 5

Curr Biol 1996
Metabolism and evolution of Haemophilus influenzae deduced from a whole-genome comparison with Escherichia coli
RL Tatusov, et al.

Science 1995
Whole-genome random sequencing and assembly of Haemophilus influenzae Rd
RD Fleischmann, et al.

Microorganisms 2019

Download Package

1 genome selected for download

Select file source

☐ All

☐ RefSeq only

☒ GenBank only

Select file types

☒ Genome sequences (FASTA)

☒ Annotation features (GTF)

☒ Annotation features (GFF)

☒ Sequence and annotation (GBFF)

☐ Transcripts (FASTA)

☒ Genomic coding sequences (FASTA)

☒ Protein (FASTA)

☒ Sequence report (JSONL)

☒ Assembly data report (JSONL)

Your selected data will be downloaded as a ZIP archive
Estimated file size is 9 MB

Name your file
GCA_000027305.1.zip

Cancel Download

- 1.2.2 Select **Download** (blue button on the top left corner) and a menu overlay should appear.

- 1.2.3 Select **All** from the Select file source section.

- 1.2.4 Check off **Genome sequences (FASTA)** Select file types section and hit the **Download** button.

This step will download a zip file to the directory of your choice. Rename this to **GCA_000027305.1** when saving. Open the archived file using 7-Zip (<https://www.7-zip.org/>) or any other available program. This will create a folder (named **ncbi_dataset** by default) with a series of subfolders.

1.2.5 Once the archive is opened, navigate to [/ncbi_dataset/data/GCA000027305.1](#) and you will find a series of files. You can transfer the following files to your **Module1/downloads** working directory:

File suffix	Description
cds_from_genomic.fna	All annotated coding sequences in nucleotide format
GCA_000027305.1_ASM2730v1_genomic.fna	Genome assembly in fasta format.
genomic.gbff	GenBank flat file format with all genomic sequences in the assembly along with metadata and annotations.
genomic.gff	Genome annotation in gff3 format.
genomic.gtf	Tab delimited file of locations and attributes for annotated features.
protein.faa	All annotated protein products in amino acid format.
sequence_report.jsonl	A JSON lines (text-based) format file used to store basic assembly statistics information.

1.3.0 Download sequencing files from the NCBI Sequence Read Archive (SRA)

Now that we've downloaded some annotations from NCBI, we are also interested in collecting the original sequencing data used to assemble the genome. We'll use this information in later sessions to dig deeper into the influenza genome. In this case we'll be downloading files in the **fastq** format.

- 1.3.1 From the Genome Assembly page in **section 1.2.1**, scroll down to [Sample details](#) section and click on the SRA entry **SRS8363689**. Click on the top entry "DNA-seq"
- 1.3.2 Alternatively, navigate to <https://www.ncbi.nlm.nih.gov/sra>. In the search bar, use the term: **SRX12843922**. This will bring you directly to the *most recent* submitted version of the KW20 whole genome sequencing.
- 1.3.3 Returning to the search bar, use the term: **SRX026485**. This will bring you directly to the *original* version of the KW20 whole genome sequencing (published 2010-09-13). You could also search for "Haemophilus influenzae KW20" and look for the oldest entry.

SRA

Full ▾

SRX026485: KW20 whole genome sequencing
1 ILLUMINA (Illumina Genome Analyzer II) run: 12M spots, 1G bases, 60+

Submitted by: University of Virginia

Study: Natural Transformation in H. Influenzae
• [SRP003474](#) • [All experiments](#) • [All runs](#)
[show Abstract](#)

Sample: Haemophilus influenzae KW20 (RR722)
[SAMN00110657](#) • [SRS114572](#) • [All experiments](#) • [All runs](#)
Organism: [Haemophilus influenzae Rd KW20](#)

Library:
Name: KW20-1
Instrument: Illumina Genome Analyzer II
Strategy: WGS
Source: GENOMIC
Selection: RANDOM
Layout: PAIRED

Spot descriptor:

1 forward 43 forward

Runs: 1 run, 12M spots, 1G bases, [604.3Mb](#)

Run	# of Spots	# of Bases	Size	Published
SRR065202	11,967,636	1G	604.3Mb	2010-09-13

Under the Runs section select the **SRR065202** hyperlink to go to the SRA information page for our genome. More information about SRA accessions can be found in **Appendix 1**.

This will take us to the SRA home page for our chosen experiment. It will look slightly different from the NCBI homepage. From this archive we will download the **fastq** files from the original sequencing run.

- 1.3.4 **OPTIONAL:** Locate the for this dataset and then click on the **FASTQ** icon in the **Download** section near the bottom. Note that this will begin downloading a ~600 Mb data file. DO NOT check the Filtered or Clipped boxes.

By default, the file will download as **SRR065202.fastq.gz**

The **fastq** file we download now will be used in later modules of the course when we return to working with the command line. You can choose to return to this step later if you wish!

Note that on the Reads and FASTA/FASTQ tab you can perform some fun filtering based on sequences. For instance, if you are only interested in a subset of data based on specific sequences (like a motif or barcode) you can try to filter exclusively for those reads. The same kind of filtering can be applied at the time of downloading your sequences.

 National Library of Medicine
National Center for Biotechnology Information

Sequence Read Archive

SearchRun BrowserAnalysesStudyProvisional SRADocumentationMirroring

Run Browser > SRR065202

KW20 whole genome sequencing (SRR065202)

MetadataAnalysisReadsData accessFASTA/FASTQ download

Download for Experiment SRX026485

Accession	Total Bases	Spots	
		Total	Filtered
☒ SRR065202	1.0G	12.0M	

Filter Runs

Search by sub-sequence, s

[What can the filter be applied to?](#)

Download

☐ Filtered ☐ Clipped or

1.4.0 Extract your downloaded files

All the files we've been downloading should now be saved into a directory such as **~/Module1/downloads**. These will need to be moved to the **Module1** analysis directory and renamed. Do this using Finder if you're on a Mac and File Explorer if you're on Windows.

IMPORTANT: Leave the **fastq** file for now because it will take some time to download.

1.4.1 Move files from your **downloads** directory (see **Section 1.2.5**) to your FGDS directory by dragging and dropping.

1.4.2 Unzip all the files (**if necessary**) in this directory. The latest download format from NCBI should not require this!

1.4.2.1 MacOS

- Select all files and double click to extract.
- Select all gz files and move them to your trash.

1.4.2.2 Windows 10/11

- Select all files > Compressed Folder Tools > Extract
- Select all gz files and move them to your trash.
- If this doesn't work on your system, you will need to download 7zip:
<https://www.wikihow.com/Extract-a-Gz-File>

1.4.3 Change all of your filenames to something more specific in your Finder/File Explorer window. Note that there are several common extensions for genomic files. A complete list is provided in **Appendix 2**.

Download name	New name	Description/format
cds_from_genomic.fna		gene file, fasta
GCA_000027305.1_ASM2730v1_genomic.fna		genome file, fasta
genomic.gbff		genbank file, text
genomic.gff		general feature format file, text
genomic.gtf		feature file, tab-delimited
protein.faa		protein file, fasta
sequence_report.jsonl		JSON lines file, text

2.0.0 Exploring and manipulating genomic data using a text editor

As we discussed at the beginning of today's class, most genomic data is going to be stored in flat text files. These are easy to access from the command line and you will learn some of the simple tools for accessing and manipulating genomic data at the command line later in this course. For now, we are going to work with text editors to access and manipulate our genomic data. For Mac, one of the best and most broadly used text editors is **BBEdit**. For Windows, **Notepad++** is often the text editor of choice.

2.0.1 Download and install the appropriate text editor for your system from one of the following links.

- MacOS: <https://www.barebones.com/products/textwrangler/download.html>
- Windows: <https://notepad-plus-plus.org/>

2.1.0 Open and explore assembly files

Open and explore all the assembly and annotation files from your **Module1** directory. This is most easily accomplished by dragging and dropping into your editing application or directly from the editing program.

2.1.1 File > Open

2.1.2 Select "Show Hidden Items" or ensure you can see "All types" of files.

2.1.3 Find the directory where you saved your *H. influenza* files and open **HinfKW20_cds.fna** and **HinfKW20_genomic.fna**.

We will now use some very simple Regular Expressions (RegEx) to search and edit some of these genomic data files. RegEx is an absurdly powerful way to perform text pattern searching and is an essential aspect of bioinformatics and nearly all aspects of computer science. We don't have time to explore RegEx in depth in this course, but we will use a few simple examples to illustrate its power. We provide more information on RegEx in **Appendix 3**.

2.2.0 Search for the beginning of new entries in a fasta file

We can count the number of contigs present in a genome assembly (ie **HinfKW20_genomic.fna**) or the number of genes in the gene file (**HinfKW20_cds.fna**) using a simple regular expression search.

2.2.1 Open up the Find/Search box (See below: Left - BBEEdit; Right – Notepad++).

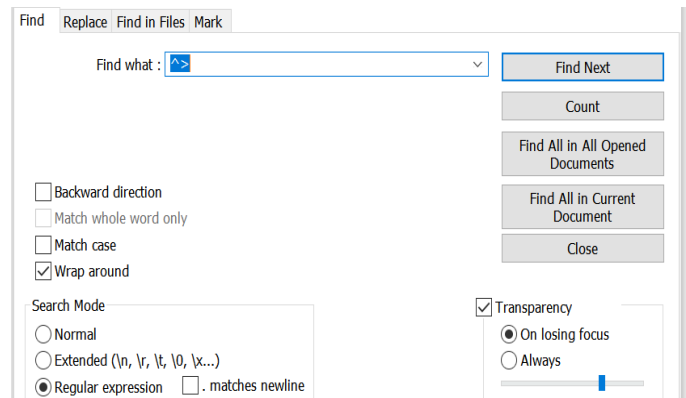
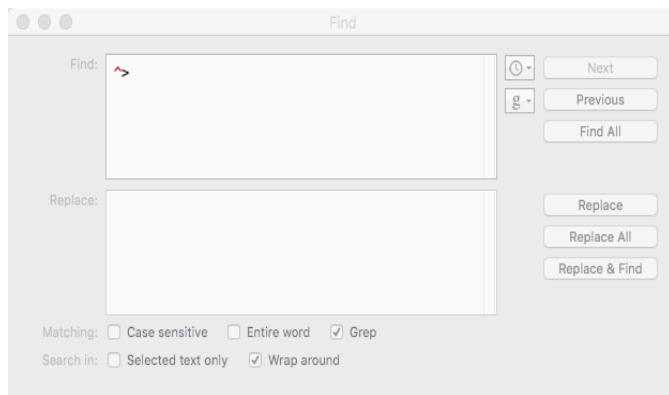
- Press Cmd + F (MacOS)
- Ctrl + F (Notepad++)

*Make sure you have selected **Grep** option in BBEEdit or **Regular Expression** option in Notepad++.*

2.2.2 Find the start of a line with the **^>**

- The **^** specifies the beginning of the line, so you are searching for all lines that start with **>**.

2.2.3 Select **Find All** (BBEEdit) or **Find All in Current Document** (Notepad ++). How many coding sequences are there in each file?



2.3.0 Count the number of genes in a gene file that use an ATG start codon

Now that we know how many entries are in our **HinfKW20_cds.fna** file we can search and see how many of those begin with an ATG start codon.

2.3.1 Find all entries with this expression: `^>.+\\nATG`

- `.` is a regular expression that means anything except for newline (ie `\\n`).
- `+` is a quantifier that means one or more times.
- `\\r\\n` and `\\n` are end-of-line (EOL) characters in Notepad++ and BBEdit, respectively. Be aware that different programs may use different end-of-line characters, and if you don't use the right one, the operation will not work. **Both MacOS X and Unix-based files use “\\n” rather than “\\r”.** If you're working on a file that was created in Windows, we strongly recommend you to change the EOL character to the Unix format. You can do this in Notepad++ by: Edit > EOL Conversion > Unix

What do our other entries look like? Are they a variant of ATG? Proceed to search for all XTG starting sequences where X could be C, G, or T.

2.3.2 Find all entries with this expression: `^>.+\\n[CGT]TG`

- `[CGT]` is used as a set to replace the “A” in ATG.
 - Another option would be to use `[^A]` to exclude A as an option but this assumes that the other character options are correctly input as C, G, or T.

Fun Fact: bacteria use multiple start codons! While we see there are 1709 entries, we find that only 1514 begin with an ATG. The remainder use either CTG, GTG, or TTG. How would you count the number for each alternative sequence?

2.4.0 Replace all start codons in a gene file with **ATG**

Suppose we wanted to replace the first codon in all our sequences with an ATG start codon sequence. We can also accomplish that with a find/replace command.

2.4.1 Use the Find window's Replace tab

2.4.2 *Find what:* `^(>.+\\n)\\w{3}`

- Again, use “\\n” instead of “\\r” for Linux/Windows.
- The parentheses around `>.+\\n` means capture this information and hold it in a temporary variable. The first set of parentheses will be held in variable `\\1`, the second set (if present) would be held in variable `\\2`, and so on.
- `\\w` matches any word character, including digits and underscores.
- `{3}` specifies the number of word characters (in this case) to match. `\\w{3}` could also have been written as `\\w\\w\\w`.

2.4.3 *Replace with:* `\\1ATG`

- `\\1` is the temporary holding variable. This will copy everything captured in the parentheses of the find statement. In this case that’s the *header* of our fasta entry that we are putting back as part of the replacement statement.
- `ATG` tells the replace statement to write “ATG”. Since you did not capture the first three characters (ie `\\w{3}`) in the find statement, this means that the replace will overwrite those characters with ATG.

2.4.4 Select the **Replace All** button. This will find and replace from the **point of the cursor** to the end of the document. Make sure your cursor is at the beginning of the document!

2.5.0 Count the number of genes ending in TAG, TGA, or TAA

2.5.1 *Find what:* `(TAG|TGA|TAA)\\n>`

- `TAG`, `TGA`, `TAA` are the variants termination codons we are interested in identifying
- `|` is the logical OR. It allows us to check for the possibility of many different regular expressions. It separates each version of the termination codon check that we are searching for.
- `\\n>` specifies the of our line followed by “>” on the next line. This ensures that we are looking at the last line of a sequence entry as it is followed immediately by a new entry.

2.5.2 Looking at the results, why don’t our counts match the total entries? Look at line 1462 for a possible answer!

2.6.0 Convert your CDS file into tab-delimited format

To solve our problem from above, we must somehow combine each fasta entry into a single line, rather than multiple ones. This should result in 1709 lines with as many entries! We’ll accomplish this by converting to a tab-delimited format which will look like: **<header>\\t<sequence>**. This will require two find/replace steps, the first to place the “\\t” between header and sequence and the second to collapse each sequence into a single line.

2.6.1 *Find what:* `(^>.+\\$)`; *Replace with:* `\\1\\t`

- `^` specifies the beginning of the line while `\\$` specifies the end of the line.
- “\\t” is a wildcard for the tab character.

2.6.2 Select the **Replace All** button.

2.6.3 *Find what:* `\\n([\\^>])`; *Replace with:* `\\1`

- `[^>]` means any character that is not `>` (not in the square bracket). Note that the `^` has a different meaning when used in the square bracket.
- This line finds every occurrence of the return character followed by anything that isn't the `>`
- Remember to replace the `\r` with `\n` when working in Linux.
- Now you replace what is found above with nothing, so all lines that aren't fasta headers (starting with the `>`) are concatenated together on one line.

2.7.0 Recount the number of genes ending in TAG, TGA, or TAA

2.7.1 *Find what:* `(TAG|TGA|TAA)$`

- `$` specifies the end of the line. We use this in favour of `\n` from section 2.5.0. Can you guess why?

2.8.0 Additional RegEx resources

These are just a couple of the manipulations that can be done with your Text Editors. Regular expressions are an incredibly useful tool for genomic data science, so try to familiarize yourself with the usage of some basic expressions in the coming weeks. Here are some resources:

- <http://www.rexegg.com/regex-quickstart.html>
- <https://regexr.com/>

3.0.0 Exploring genomic data with the Integrative Genomics Viewer (IGV)

By now your **fastq** file should have finished downloading. Let's move that to the **Module1** directory and rename it. We won't need to complete this step until we move to the command line lectures so you can keep it in the compressed format. Note that the unzipped file will actually take quite a bit of space: 4.6 Gb uncompressed!

3.0.1 Move the file by dragging it from your **Downloads** directory to your **Module1** directory.

3.0.2 Unzip the file (later!)

5.0.2.1 MacOS

- Select all files and double click to extract.
- Select all gz files and move them to your trash.

5.0.2.2 Windows 10/11

- Select all files > Compressed Folder Tools > Extract
- Select all gz files and move them to your trash.
- Use 7zip if necessary.

3.0.3 Rename the file **sra_data.fastq.gz**

Download name	New name	Description/format
sra_data.fastq.gz	HinfKW20_data.fastq	gene file, fasta

3.1.0 Download and install the Integrative Genomics Viewer

3.1.1 You can download IGV from the Broad Institute at the following link:

<https://software.broadinstitute.org/software/igv/download>

Make sure to choose the correct version for your operating system. If you already have Java installed (version 21 or later) then you can download the version which does not have "Java included".

Extensive information about the IGV can be found at <http://software.broadinstitute.org/software/igv/UserGuide>.

3.1.2 **If IGV does not open properly**, you likely have an incompatible version of Java. Uninstall your version of IGV and redownload/install using the "Java included" version.

3.2.0 Use IGV to explore genome-sequencing data with alignment files

IGV allows you to import and explore genomic data from flat text files in a user-friendly format. Today we will be looking at a reference alignment file to learn some of the basic functionality of IGV. You will need to download two additional files from the Quercus website (Module1 > Data) for this section. These files were generated by aligning the reads from our fastq file to the *H. influenzae* KW20 reference genome.

- Sorted BAM alignment file – 2024_HinfKW20.sorted.bam
- Index for the sorted BAM alignment file – 2024_HinfKW20.sorted.bam.bai

3.3.0 Open IGV and generate a genome file from a reference genome

Now that we have the associated .bam and .bam.bai files for our genome, we can use IGV to actually look at the reference sequence and how the reads from our downloaded SRA fastq files align to the genome. We'll be using the H. influenzae KW20 reference data that we've been collecting throughout the module.

3.3.1 Open IGV.

3.3.2 Load a genome file using **Genomes > Load Genome from File...**

3.3.3 From your data directory select [redacted] and then select Open

This action will load the initial genome information from our data and we'll see the contig name for our genome (L42023.1)

3.3.4 Load the gene names and locations using **File > Load from File...**

3.3.5 From the data directory select [redacted]

This will load a new track *onto* your loaded genome that will show the various open reading frames in the genome. Remember that a .gff file is a *general feature format* so we've basically annotated our genome with the gene locations!

3.4.0 Load and explore the reference alignment that you've downloaded in IGV

Next, we will use that .bam file we downloaded from Quercus to annotate the location of all the sequencing reads from that very large fastq we downloaded from the SRA. The .bam file was made using a program called **samtools**, where we generated an annotation of the fastq reads and by aligning them within our reference genome (aka **HinfKW20_genomic.fna**). We'll learn more about how to do this in later sessions! For now, we'll load the alignment information.

3.4.1 **File > Load From File...**

3.4.2 Select the sorted bam file [redacted]

3.5.0 Review some basic tools that you can use to explore the alignment in IGV

3.5.1 Identify the displayed tracks by looking at the information presented in the left pane.

3.5.2 Right click on a track to expand it.

3.5.3 Zoom in and out with the (+) and (-) zoom slider.

3.5.4 Shift the window by clicking and dragging.

3.5.5 Scroll over elements to obtain more information about the region, read, or gene. You can modify this function with the comment symbol in the toolbar.

3.5.6 Right-click to bring up context-sensitive options for displaying or analyzing the data.

3.5.7 Search for specific regions in the search box (contig: baseStart-baseEnd).

3.5.8 Find examples of locations where reads identify different bases (coloured letters), where there are indels (solid line, I), and where reads can be mapped to multiple locations (white fill).

3.6.0 Find a particular motif in the reference genome

3.6.1 **Tools > Find Motif...**

3.6.2 Enter sequence and track names and select OK.

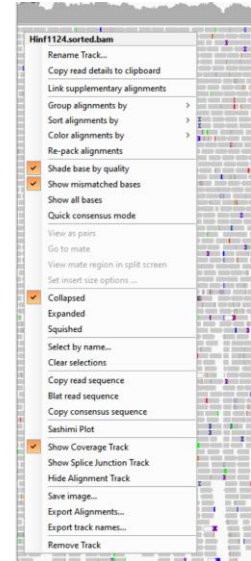
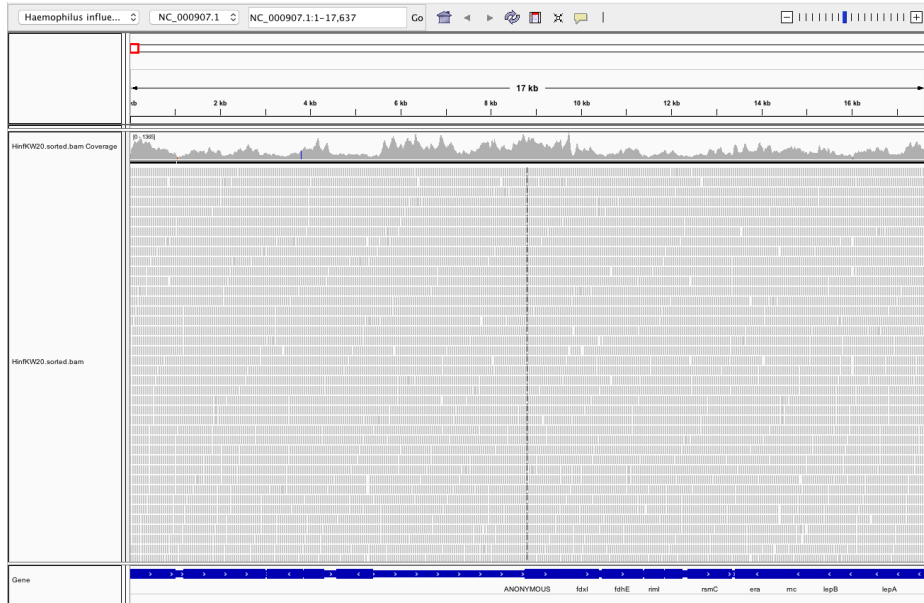
3.6.3 Remove the tracks by left clicking and selecting “Remove Tracks”.

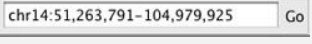
3.7.0 Saving your session in its current state

3.7.1 **File > Save Session...**

3.7.2 Name the session and click OK.

3.7.3 Reopen the session with **File > Open Session...**



	Genome drop-down box: Loads a genome. more...
	Chromosome drop-down box: Zooms to a chromosome. more...
	Search box: Displays the chromosome location being shown. To scroll to a different location, enter the gene name, locus, or track name and click Go. more...
	Whole genome view: Zooms to whole genome view. more...
	Moves backward and forward through views of the genome like the back and forward buttons in a web browser.
	Refreshes the display.
	Defines a region of interest on the chromosome. more...
	Reduces the row height on all tracks to fit all data for the region in view into the window; will also expand tracks (to their maximum preferred size) to fill the view, if needed.
	Controls the popup information behavior. Options include displaying the information as the cursor hovers over an item, or when the item is clicked. The popup can also be disabled.
	Zoom Slider: Zooms in and out on a chromosome. more...

Quick visual reference of IGV menu-bar tools

4.0.0 Class summary

That concludes our first lecture and introduction to working with genomic data. Altogether we've explored the following:

- Common sequencing and data file types.
- Navigating the NCBI to download genome assembly data.
- Navigating the Sequence Read Archive to download raw sequencing data.
- Using regular expressions to search and replace data.
- Visualizing genomes with the Integrative Genomics Viewer.

4.1.0 Post-lecture assessment (9% of final grade)

Soon after this lecture, a homework assignment will be made available on Quercus in the assignment section. It will build on the ideas and/or data generated within this lecture. Each homework assignment will be worth 9% of your final mark. If you have assignment-related questions, please try the following steps in the order presented:

- Check the internet for a solution – read forums and learn to navigate for answers.
- Generate a discussion on Quercus outlining what you've tried so far and see if other students can contribute to a solution.
- Contact course teaching assistants or the instructor.

4.2.0 Suggested class preparation for Module 2

Next week we will begin exploring the Galaxy interface and learning how to perform quality control and genome assembly on data from sources like the SRA. To prepare for this, we suggest the following Galaxy Tutorial:

- Galaxy 101 for everyone:
 - <https://gallantries.github.io/video-library/videos/introduction/galaxy-intro-101-everyone/tutorial/>
 - This is a ~1.25h video tutorial that you can watch/listen to ahead of time to familiarize yourself with the ideas of Galaxy.
 - We will go over much of this in class but the focus will be on navigating and using Galaxy.

4.3.0 Acknowledgements

This course was originally conceived and produced by Dr. David Guttman and Dr. Marcus Dillon.

- **Revision 1.0.0:** edited and prepared by David Guttman, Ph.D. and Marcus Dillon, Ph.D.
- **Revision 2.0.0:** edited and prepared for *CSB1021HF LEC0131, 10-2021* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.1:** edited and prepared for *CSB1021HF LEC0131, 10-2023* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.2:** edited and prepared for *CSB1021HF LEC0131, 10-2024* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.3:** edited and prepared for *CSB1021HF LEC0131, 10-2025* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.

5.0.0 Appendix 1: SRA accessions

Accession Prefix	Accession Name	Definition	Example
SRA	SRA submission accession	The submission accession represents a virtual container that holds the objects represented by the other five accessions and is used to track the submission in the archive.	Since the SRA accession number is an artificial packaging construct, there is no example available since the SRA accession number has no specific response page
SRP	SRA study accession	A Study is an object that contains the project metadata describing a sequencing study or project. Imported from BioProject.	HTML
SRX	SRA experiment accession	An Experiment is an object that contains the metadata describing the library, platform selection, and processing parameters involved in a particular sequencing experiment.	HTML
SRR	SRA run accession	A Run is an object that contains actual sequencing data for a particular sequencing experiment. Experiments may contain many Runs depending on the number of sequencing instrument runs that were needed.	HTML
SRS	SRA sample accession	A Sample is an object that contains the metadata describing the physical sample upon which a sequencing experiment was performed. Imported from BioSample.	HTML
SRZ	SRA analysis accession	An analysis is an object that contains a sequence data analysis BAM file and the metadata describing the sequence analysis.	

The first letter of the accession prefix shows which [INSDC](#) archive the data originated from (S = [NCBI](#)-SRA, E = [EMBL](#)-SRA, D = [DDBJ](#)-SRA):

Data originating from EMBL-EBI:

Accession Prefix	Accession Name
ERA	ERA submission accession
ERP	ERA study accession
ERX	ERA experiment accession
ERR	ERA run accession
ERS	ERA sample accession
ERZ	ERA analysis accession

Data originating from DDBJ:

Accession Prefix	Definition
DRA	DRA submission accession
DRP	DRA study accession
DRX	DRA experiment accession
DRR	DRA run accession
DRS	DRA sample accession
DRZ	DRA analysis accession

More info at: <https://www.ncbi.nlm.nih.gov/books/NBK56913/>

- search for “What do the different SRA accessions represent?”

6.0.0 Appendix 2: Genomic file extensions

File Extension	File Type
.asn	genome record in asn.1 format
.faa	protein sequences in fasta format, text file
.ffn	protein coding portions of the genome segments
.fna	genome fasta sequence
.frn	rna coding portions of the genome segments
.gb	Genbank
.gbk	genome in Genbank file format
.gff	General file format used for storing annotations like genome features
.gpff	Genbank protein
.ptt	protein table
.rnt	rna table
.rpt	summary report
.val	binary file (genome project)
.tar	TAR archive, a common Linux archive file format.
.gz	gzip, a compressed format. Not the same as .zip
.tar.gz	gzipped tar, usually a tar file that was subsequently gzipped
.tgz	gzipped tar, usually gzipped by the tar application
.zip	Zip, a common Windows compression format

Additional sources:

<https://bioinformatics.uconn.edu/resources-and-events/tutorials-2/file-formats-tutorial/#>

7.0.0 Appendix 3: Regular expressions cheat sheet



Anchors		Sample Patterns																																																																				
<code>^</code>	Start of line +	<code>([A-Za-z0-9-]+)</code>	Letters, numbers and hyphens																																																																			
<code>\A</code>	Start of string +	<code>(\d{1,2}\V\d{1,2}\V\d{4})</code>	Date (e.g. 21/3/2006)																																																																			
<code>\$</code>	End of line +	<code>([\^s]+(?:=\.(jpg gif png))\.\2)</code>	jpg, gif or png image																																																																			
<code>\Z</code>	End of string +	<code>(^[1-9]{1}\$ ^1-4]{1}[0-9]{1}\$ ^50\$)</code>	Any number from 1 to 50 inclusive																																																																			
<code>\b</code>	Word boundary +	<code>(#?([A-Fa-f0-9]){3}([A-Fa-f0-9]){3})?)</code>	Valid hexadecimal colour code																																																																			
<code>\B</code>	Not word boundary +	<code>((?=[\d])(?=[a-z])(?=[A-Z]).{8,15})</code>	8 to 15 character string with at least one upper case letter, one lower case letter, and one digit (useful for passwords).																																																																			
<code>\<</code>	Start of word	<code>(\w+@[a-zA-Z_]+?\.[a-zA-Z]{2,6})</code>	Email addresses																																																																			
<code>\></code>	End of word	<code>(\<\/?[^\>]+\>)</code>	HTML Tags																																																																			
Character Classes		Note These patterns are intended for reference purposes and have not been extensively tested. Please use with caution and test thoroughly before use.																																																																				
<code>\c</code>	Control character																																																																					
<code>\s</code>	White space																																																																					
<code>\S</code>	Not white space																																																																					
<code>\d</code>	Digit																																																																					
<code>\D</code>	Not digit	Quantifiers		Ranges																																																																		
<code>\w</code>	Word																																																																					
<code>\W</code>	Not word																																																																					
<code>\xhh</code>	Hexadecimal character hh																																																																					
<code>\Oxxx</code>	Octal character xxx																																																																					
POSIX Character Classes		Special Characters <table><tr><td><code>\</code></td><td>Escape Character +</td></tr><tr><td><code>\n</code></td><td>New line +</td></tr><tr><td><code>\r</code></td><td>Carriage return +</td></tr><tr><td><code>\t</code></td><td>Tab +</td></tr><tr><td><code>\v</code></td><td>Vertical tab +</td></tr><tr><td><code>\f</code></td><td>Form feed +</td></tr><tr><td><code>\a</code></td><td>Alarm</td></tr><tr><td><code>[\b]</code></td><td>Backspace</td></tr><tr><td><code>\e</code></td><td>Escape</td></tr><tr><td><code>\N{name}</code></td><td>Named Character</td></tr></table> String Replacement (Backreferences) <table><tr><td><code>\$n</code></td><td>nth non-passive group</td></tr><tr><td><code>\$2</code></td><td>"xyz" in /^(abc(xyz))\$/</td></tr><tr><td><code>\$1</code></td><td>"xyz" in /^(?:abc)(xyz)\$/</td></tr><tr><td><code>\$`</code></td><td>Before matched string</td></tr><tr><td><code>\$'</code></td><td>After matched string</td></tr><tr><td><code>\$+</code></td><td>Last matched string</td></tr><tr><td><code>\$&</code></td><td>Entire matched string</td></tr><tr><td><code>\$_</code></td><td>Entire input string</td></tr><tr><td><code>\$</code></td><td>Literal "\$"</td></tr></table> Metacharacters (must be escaped) <table><tr><td><code>^</code></td><td><code>[</code></td><td><code>.</code></td></tr><tr><td><code>\$</code></td><td><code>{</code></td><td><code>*</code></td></tr><tr><td><code>(</code></td><td><code>\</code></td><td><code>+</code></td></tr><tr><td><code>)</code></td><td><code> </code></td><td><code>?</code></td></tr><tr><td><code><</code></td><td><code>></code></td><td></td></tr></table> Note Ranges are inclusive. Pattern Modifiers <table><tr><td><code>g</code></td><td>Global match</td></tr><tr><td><code>i</code></td><td>Case-insensitive</td></tr><tr><td><code>m</code></td><td>Multiple lines</td></tr><tr><td><code>s</code></td><td>Treat string as single line</td></tr><tr><td><code>x</code></td><td>Allow comments and white space in pattern</td></tr><tr><td><code>e</code></td><td>Evaluate replacement</td></tr><tr><td><code>U</code></td><td>Ungreedy pattern</td></tr></table>		<code>\</code>	Escape Character +	<code>\n</code>	New line +	<code>\r</code>	Carriage return +	<code>\t</code>	Tab +	<code>\v</code>	Vertical tab +	<code>\f</code>	Form feed +	<code>\a</code>	Alarm	<code>[\b]</code>	Backspace	<code>\e</code>	Escape	<code>\N{name}</code>	Named Character	<code>\$n</code>	nth non-passive group	<code>\$2</code>	"xyz" in /^(abc(xyz))\$/	<code>\$1</code>	"xyz" in /^(?:abc)(xyz)\$/	<code>\$`</code>	Before matched string	<code>\$'</code>	After matched string	<code>\$+</code>	Last matched string	<code>\$&</code>	Entire matched string	<code>\$_</code>	Entire input string	<code>\$</code>	Literal "\$"	<code>^</code>	<code>[</code>	<code>.</code>	<code>\$</code>	<code>{</code>	<code>*</code>	<code>(</code>	<code>\</code>	<code>+</code>	<code>)</code>	<code> </code>	<code>?</code>	<code><</code>	<code>></code>		<code>g</code>	Global match	<code>i</code>	Case-insensitive	<code>m</code>	Multiple lines	<code>s</code>	Treat string as single line	<code>x</code>	Allow comments and white space in pattern	<code>e</code>	Evaluate replacement	<code>U</code>	Ungreedy pattern
<code>\</code>	Escape Character +																																																																					
<code>\n</code>	New line +																																																																					
<code>\r</code>	Carriage return +																																																																					
<code>\t</code>	Tab +																																																																					
<code>\v</code>	Vertical tab +																																																																					
<code>\f</code>	Form feed +																																																																					
<code>\a</code>	Alarm																																																																					
<code>[\b]</code>	Backspace																																																																					
<code>\e</code>	Escape																																																																					
<code>\N{name}</code>	Named Character																																																																					
<code>\$n</code>	nth non-passive group																																																																					
<code>\$2</code>	"xyz" in /^(abc(xyz))\$/																																																																					
<code>\$1</code>	"xyz" in /^(?:abc)(xyz)\$/																																																																					
<code>\$`</code>	Before matched string																																																																					
<code>\$'</code>	After matched string																																																																					
<code>\$+</code>	Last matched string																																																																					
<code>\$&</code>	Entire matched string																																																																					
<code>\$_</code>	Entire input string																																																																					
<code>\$</code>	Literal "\$"																																																																					
<code>^</code>	<code>[</code>	<code>.</code>																																																																				
<code>\$</code>	<code>{</code>	<code>*</code>																																																																				
<code>(</code>	<code>\</code>	<code>+</code>																																																																				
<code>)</code>	<code> </code>	<code>?</code>																																																																				
<code><</code>	<code>></code>																																																																					
<code>g</code>	Global match																																																																					
<code>i</code>	Case-insensitive																																																																					
<code>m</code>	Multiple lines																																																																					
<code>s</code>	Treat string as single line																																																																					
<code>x</code>	Allow comments and white space in pattern																																																																					
<code>e</code>	Evaluate replacement																																																																					
<code>U</code>	Ungreedy pattern																																																																					
<code>[[:upper:]]</code>	Upper case letters																																																																					
<code>[[:lower:]]</code>	Lower case letters																																																																					
<code>[[:alpha:]]</code>	All letters																																																																					
<code>[[:alnum:]]</code>	Digits and letters																																																																					
<code>[[:digit:]]</code>	Digits																																																																					
<code>[[:xdigit:]]</code>	Hexadecimal digits																																																																					
<code>[[:punct:]]</code>	Punctuation																																																																					
<code>[[:blank:]]</code>	Space and tab																																																																					
<code>[[:space:]]</code>	Blank characters																																																																					
<code>[[:cntrl:]]</code>	Control characters																																																																					
<code>[[:graph:]]</code>	Printed characters																																																																					
<code>[[:print:]]</code>	Printed characters and spaces																																																																					
<code>[[:word:]]</code>	Digits, letters and underscore																																																																					
Assertions		String Replacement (Backreferences) <table><tr><td><code>\$n</code></td><td>nth non-passive group</td></tr><tr><td><code>\$2</code></td><td>"xyz" in /^(abc(xyz))\$/</td></tr><tr><td><code>\$1</code></td><td>"xyz" in /^(?:abc)(xyz)\$/</td></tr><tr><td><code>\$`</code></td><td>Before matched string</td></tr><tr><td><code>\$'</code></td><td>After matched string</td></tr><tr><td><code>\$+</code></td><td>Last matched string</td></tr><tr><td><code>\$&</code></td><td>Entire matched string</td></tr><tr><td><code>\$_</code></td><td>Entire input string</td></tr><tr><td><code>\$</code></td><td>Literal "\$"</td></tr></table> Metacharacters (must be escaped) <table><tr><td><code>^</code></td><td><code>[</code></td><td><code>.</code></td></tr><tr><td><code>\$</code></td><td><code>{</code></td><td><code>*</code></td></tr><tr><td><code>(</code></td><td><code>\</code></td><td><code>+</code></td></tr><tr><td><code>)</code></td><td><code> </code></td><td><code>?</code></td></tr><tr><td><code><</code></td><td><code>></code></td><td></td></tr></table> Note Items marked + should work in most regular expression implementations.		<code>\$n</code>	nth non-passive group	<code>\$2</code>	"xyz" in /^(abc(xyz))\$/	<code>\$1</code>	"xyz" in /^(?:abc)(xyz)\$/	<code>\$`</code>	Before matched string	<code>\$'</code>	After matched string	<code>\$+</code>	Last matched string	<code>\$&</code>	Entire matched string	<code>\$_</code>	Entire input string	<code>\$</code>	Literal "\$"	<code>^</code>	<code>[</code>	<code>.</code>	<code>\$</code>	<code>{</code>	<code>*</code>	<code>(</code>	<code>\</code>	<code>+</code>	<code>)</code>	<code> </code>	<code>?</code>	<code><</code>	<code>></code>																																			
<code>\$n</code>	nth non-passive group																																																																					
<code>\$2</code>	"xyz" in /^(abc(xyz))\$/																																																																					
<code>\$1</code>	"xyz" in /^(?:abc)(xyz)\$/																																																																					
<code>\$`</code>	Before matched string																																																																					
<code>\$'</code>	After matched string																																																																					
<code>\$+</code>	Last matched string																																																																					
<code>\$&</code>	Entire matched string																																																																					
<code>\$_</code>	Entire input string																																																																					
<code>\$</code>	Literal "\$"																																																																					
<code>^</code>	<code>[</code>	<code>.</code>																																																																				
<code>\$</code>	<code>{</code>	<code>*</code>																																																																				
<code>(</code>	<code>\</code>	<code>+</code>																																																																				
<code>)</code>	<code> </code>	<code>?</code>																																																																				
<code><</code>	<code>></code>																																																																					
<code>?=</code>	Lookahead assertion +																																																																					
<code>?!</code>	Negative lookahead +																																																																					
<code>?<=</code>	Lookbehind assertion +																																																																					
<code>?!= or ?<!</code>	Negative lookbehind +																																																																					
<code>?></code>	Once-only Subexpression																																																																					
<code>?()</code>	Condition [if then]																																																																					
<code>?() </code>	Condition [if then else]																																																																					
<code>?#</code>	Comment																																																																					
Note		Available free from AddedBytes.com																																																																				

